



Grado en Ingeniería Informática

Sistemas Distribuidos 25/26

Grupo 81

Proyecto final

«Servicio de envío de mensajes»

Chen Xu — 100522395

Miguel Amiel Gil — 100525454

Profesor

Felix García Carballeira

Tabla de contenidos

1. Descripción del código	2
1.1. Introducción	2
1.2. Diseño	2
1.3. Implementación de las peticiones	4
1.4. Protocolo para GETFILE	5
2. Compilación y obtención de los ejecutables	7
2.1. Ejecución del proyecto	7
2.2. Makefile de RPC	8
3. Pruebas realizadas	9
3.1. Pruebas que simulan flujo habitual	9
3.1.1. Resultados en clientes	9
3.1.2. Resultados en servidores	11
3.2. Pruebas de comprobación de errores	12
4. Conclusiones	14

1. Descripción del código

1.1. Introducción

Para el enunciado de la parte 1 del enunciado de la práctica de servicio de envío de mensajes se han implementado la comunicación y los distintos tipos de servicios pedidos.

1.2. Diseño

Para implementar los servicios requeridos, se ha seguido el protocolo especificado en el guión de la práctica. Nuestra implementación esta organizada con el siguiente sistema de directorios para facilitar la organización de la información.

- **Directorio raíz del proyecto**
 - Makefile
 - README.md
 - **/src**
 - **/RPC_server**
 - Makefile
 - proxy-rpc.c
 - server-rpc.c
 - archivos generados por rpc-gen
 - **/servicio_web_final**
 - espacios_clnt.py
 - servicio_web_espacios.py
 - servidor.c
 - lines.c
 - client.py
 - gestionar_peticiones.c
 - **/include**
 - gestionar_peticiones.h
 - lines.h
 - proxy-rpc.h

En el directorio «src» se localizan los archivos que incluyen el código que incorpora las funcionalidades del sistema. En el directorio «include» se guardan los headers con las definiciones de las funciones de «src». Por un lado, se ha implementado un servidor concurrente que gestiona las peticiones asociadas a las acciones deseadas por el cliente mediante una pool de threads. Esta pool recibe el socket del cliente que ha enviado la petición y, en función de la instrucción recibida, se llama a una función concreta del fichero gestionar_peticiones. El servidor almacena la información de cada usuario en un directorio que se llenará de archivos que poseen el mismo nombre que el propio usuario. Estos archivos cliente se encuentran almacenados en el directorio llamado «clientes».

Por otro lado, se han implementado un servidor RPC y servidor web para realizar la parte 2 de la práctica. El servidor RPC recibe por parte del servidor central cada una de las peticiones que realizan los clientes y las imprime. Este servidor RPC fue implementado utilizando ONC RPC y ejecutando rpcgen.

En cambio, el servidor web recibe cada uno de los mensajes que el usuario desea enviar y lo normaliza, eliminando espacios redundantes. Este servidor web fue implementado mediante la utilización de Flask. Hemos usado REST y solo hay 1 tipo de solicitud POST que se usa cada vez que llega una petición al servidor.

Para evitar problemas de concurrencia en los archivos, se han usado flocks que permiten asegurar que un archivo no es leído mientras está siendo escrito o que varios clientes escriban a la vez sobre el mismo. Es importante mencionar, como se almacena la información en cada uno de estos archivos. En los primeros bytes del archivo se almacena un **struct info_usuario** que nos permite conservar el estado de cada uno de los usuarios.

El **struct info usuario** almacena la siguiente información:

1. nombre_cliente -> almacena el nombre de usuario del cliente para poder acceder a este de forma sencilla.
2. estado -> un número que indica si el usuario asociado está conectado o no. Concretamente, toma el valor 0 si el usuario no está conectado y 1 si lo está.
3. ip -> la ip del cliente asociado en formato string.
4. puerto_escucha_cliente -> almacena el puerto de escucha del thread worker que se encarga de las acciones que no realiza el usuario. Por ejemplo, la llegada de un mensaje o la transferencia del archivo pedido por otro usuario.

Además se utilizaron mutex y una variable de condición global para asegurar que las peticiones no se sobrescribiesen antes de que lleguen al pool de threads. Una vez que la información de la petición se pasa al thread, se puede pasar a tratar la siguiente petición.

Para leer y escribir en archivos se añadieron dos nuevas funciones al archivo lines.c que se dió en clase. Estas funciones intentan leer o escribir en un archivo de forma continuada hasta que se hayan procesado los bytes especificados. Es similar a las funciones inicialmente dadas pero se han modificado para que la lectura siga a pesar de encontrar un salto de línea o el terminador «\0».

Para gestionar el envío de mensajes se añadió un struct llamado «mensaje» con la siguiente estructura:

1. usuario_origen -> almacena el nombre del cliente remitente.
2. texto_mensaje -> almacena el mensaje que se desea enviar.
3. nombre_fichero -> almacena el nombre del fichero especificado por el usuario (solo se usa para **SENDATTACH**)

1.3. Implementación de las peticiones

En las siguientes líneas se explican, de forma resumida, la implementación de cada una de las peticiones.

REGISTER: Esta petición permite registrar un nombre de usuario dado. En nuestra implementación, tras recibir el nombre de usuario, abrimos el directorio clientes (donde guardamos los datos de todos los usuarios). Si este no existe, lo creamos. Una vez creado, abrimos/creamos el archivo asociado a ese cliente en concreto. Si este archivo ya existía significa que el cliente ya estaba registrado, Por lo tanto, damos un error(codigo = 1). También se devuelven códigos de error si hay algún error durante la modificación del archivo del cliente(codigo = 2). Inicializamos el struct info_usuario con nombre_cliente = userName, estado = 0 (inicialmente desconectado) , ip vacía y puerto con valor default 0.

UNREGISTER: Esta petición permite borrar el registro de un nombre de un usuario dado. Para ello, se comprueba si el archivo existe. Si el usuario no existía se devuelve error (codigo = 1). Además, si ya hay algún usuario utilizando el archivo, generamos un error (codigo = 2). Si no hay ningún error de los anteriormente mencionados, se procede a borrar el archivo mediante la utilización de remove(ruta_archivo).

CONNECT: Esta petición permite modificar el estado de conexión del usuario, estableciéndolo como CONECTADO. El cliente crea un thread Para ello, recibimos el nombre de usuario y el puerto del thread worker del cliente. Si hay un error durante la obtención del puerto o se envía un puerto no válido se da un error(codigo = 3). Durante el connect también se guarda en el archivo la dirección IP del cliente. Esta se ha obtenido al realizar el accept por parte del servidor. Una vez obtenidos los datos y tras haber comprobado que son correctos leemos el archivo del usuario indicado y fijamos el atributo estado del struct info_usuario a 1.

DISCONNECT: Esta petición permite modificar el estado del usuario de conectado a desconectado. Como se explicó anteriormente, en los ficheros de los clientes se guarda un struct con el estado del cliente que puede ser 1 o 0. En nuestra implementación, el servidor buscará el archivo con el nombre del cliente y leerá el struct guardado. Si el estado extraído es 1, este se cambiará a 0 y se sobrescribirá el struct anterior.

USERS: Esta petición le permite al usuario solicitante obtener una lista de los usuarios conectados. En nuestra implementación, en el servidor se abre el directorio de los clientes y se revisa el estado de cada cliente uno por uno leyendo el struct guardado.

Si el cliente está conectado, se le añade a una lista de nombres con `strdup` y se le suma 1 al número de usuarios conectados. Una vez se hayan leído todos los archivos, se devuelve la información al usuario como se especifica en el protocolo. Para hacer la modificación pedida en la parte 2, usamos un `malloc` y `sprintf` para crear el array para guardar el nombre, ip y puerto en una cadena de caracteres que después se guardan en otro array de usuarios. El intercambio de los mensajes se hace como se especifica en el protocolo de la guía.

SEND: Esta petición le permite a los usuarios mandarle mensajes a otros usuarios. En nuestra implementación, el servidor lee el struct guardado del usuario y extrae la ip y el puerto de su thread worker para mandar el mensaje. Primero, el servidor prepara el mensaje a enviar asignandole el id y rellenando el struct. Después crea un socket que se conecta con el socket del thread worker destinatario y le manda el mensaje. Finalmente se crea otro socket distinto que se conecta con el worker del remitente para mandar el mensaje de «SEND_MESS_ACK» para avisar de que el mensaje se ha mandado correctamente. Además, si el usuario destinatario no está conectado en ese momento, se guarda en el archivo del usuario. De esta manera, cuando el usuario se vuelva a conectar, se cambiará su estado sobreescribiendo el struct y como el puntero apunta a después del struct, podemos leer los mensajes que guardamos al final del archivo.

SENDATTACH: Esta petición es idéntica al **SEND** descrito anteriormente. Su único cambio es que en esta petición, el struct de mensaje rellenará el último campo, `nombre_fichero`, con el nombre de fichero que especifique el usuario.

GETFILE: Esta petición permite recuperar un archivo perteneciente a un usuario concreto. El remitente debe especificar la ruta del archivo original en el otro cliente y la ruta local donde desea que se guarde la información. Para almacenar los datos de los clientes conectados que se requieren para esta funcionalidad, se ha utilizado un diccionario. Este posee como llave el nombre de usuario de cada cliente. El valor asociado a cada key corresponde a una tupla formada por la ip y puerto de cada cliente. En el **GETFILE** se crea un archivo en la ruta local especificada y se escribe la información del archivo del remitente a medida que se recibe. Una vez escrito el archivo en su totalidad, cerramos el descriptor. Se manda y recibe los contenidos del archivo en bloques de 1024 bytes.

1.4. Protocolo para GETFILE

Cuando el archivo no existe en el directorio del usuario al que se le pide el archivo, se devolverá código de operación 2 al usuario que inicia la petición y se verá en la terminal el siguiente mensaje:

- **FILE TRANSFER FAIL, specified fileName does not exist**

En el caso de cualquier otro error, como fallo al abrir el archivo o fallo en la transferencia del archivo se devolverá código de operación 1 y se verá en la terminal el siguiente mensaje:

- **FILE TRANSFER FAIL, system failed**

En el caso de que la operación haya sido exitosa, se devolverá 0 y se imprimirá el siguiente mensaje en la terminal:

- **FILE TRANSFER OK**

Para el cliente que recibe la petición, cuando se acepta la conexión y recibe la petición de «GET_FILE» se verá en la terminal:

- **FILE <nombre del archivo> REQUESTED BY <nombre del solicitante>**

En el caso de que haya habido algún error en la transferencia se verá el siguiente mensaje:

- **FILE TRANSFER FAILED<excepción>**

2. Compilación y obtención de los ejecutables

2.1. Ejecución del proyecto

Para compilar nuestro proyecto, se debe acudir al directorio raíz del proyecto y ejecutar al Makefile. Este Makefile llama al Makefile del servidor RPC. Por otro lado, enlazamos los archivos objeto de gestionar_peticiones y lines con el archivo objeto del servidor para crear el ejecutable final. Concretamente, para conseguir ejecutar el sistema de forma correcta, se deben ejecutar los siguientes comandos:

1. Ejecutar el Makefile del directorio raíz del proyecto

```
make
```

2. Ejecutamos el servidor web, encargado de realizar la normalización de los espacios de los mensajes intercambiados.

```
cd src/servicio_web_final
python3 servicio_web_espacios.py
```

3. Ejecutamos el servidor RPC

```
cd src/RPC_server
./server-rpc
```

4. Ejecutamos el servidor que gestiona las peticiones del cliente(REGISTER, CONNECT...). Se utiliza la variable de entorno LOG_RPC_IP.

```
LOG_RPC_IP=localhost ./servidor -p 5000
```

5. Ejecutamos el número de terminales deseadas, cada una de ellas asociadas a un cliente concreto.

```
cd src
python3 client.py -s localhost -p 5000
```

Cada terminal será utilizada por un único cliente. En ella podrá realizar todas las peticiones deseadas. El servidor procesará las peticiones y devolverá la respuesta concreta.

Con el proyecto, se ha proporcionado un archivo run.sh. Este archivo crea 5 terminales. El objetivo es facilitar la simulación del sistema. Se asocia una terminal al servidor que gestiona las peticiones, otra terminal al servidor web que normaliza los mensajes y otra terminal al servidor RPC que registra cada una de las peticiones realizadas. Por otro lado, se abren dos terminales que ejecutan el archivo client.py. A través de estas terminales, se pueden enviar peticiones para simular el uso del sistema. Si se considera conveniente, se pueden abrir más terminales para incluir más clientes en la ejecución. Este archivo .sh permite comprobar el flujo utilizando las variables **IP_SERVER_PETICIONES** y **PUERTO_SERVER_PETICIONES**. Estas variables modifican donde escucha y se ejecuta el servidor.

Para ejecutarlo se debe introducir el siguiente comando desde el directorio raíz del proyecto, en una terminal:

```
chmod +x run.sh
./run.sh
```


2.2. Makefile de RPC

Dentro de la carpeta `RPC_server` se encuentra otro Makefile, el cual es llamado desde el Makefile general. Este se encarga de crear los archivos objeto tanto de la parte cliente y servidor de ONC RPC. Concretamente, se crean los archivos objeto de los siguientes archivos: `RPC_ampliacion_svc`, `RPC_ampliacion_xdr`, `RPC_aplicacion_clnt`, `proxy-rpc` y `server-rpc`.

Una vez creados los archivos objetos, creamos una librería dinámica a partir del archivo `proxy-rpc`. De esta manera, el servidor que gestiona las peticiones importa dicho proxy y ejecuta las llamadas de la misma. AL llamar al proxy, la instrucción realizada se envía al servidor RPC.

3. Pruebas realizadas

Las pruebas se realizan una vez que todos los clientes y servidores están ya en ejecución.

3.1. Pruebas que simulan flujo habitual

En este apartado se realizarán pruebas que emulan un flujo sin errores del código. Estas pruebas se hicieron en la misma red con dos dispositivos, un cliente en cada uno, para comprobar la conexión.

3.1.1. Resultados en clientes

En esta sección, mostramos la ejecución de tests cliente que se realizan entre dos dispositivos con dos IPs distintas. De esta forma, podemos simular un flujo de mensajes natural entre clientes distintos.

ID	Descripción	Terminal Cli1	Terminal Cli2
1	Ejecución de flujo sin errores del servicio	<pre> c> REGISTER chen REGISTER OK c> CONNECT chen CONNECT OK c> USERS CONNECTED USERS (1 users connected) OK chen::127.0.0.1::48499 c> SEND miguel hola que ase mondongo SEND OK - MESSAGE 0 c> SEND MESSAGE 0 OK GETFILE MESSAGE 1 FROM miguel Toma mi archivo END FILE /home/miguel_a/ Documents/UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt c> GETFILE miguel /home/ miguel_a/Documents/ UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt /home/ chenpower3/Desktop/ distribuidos/final/ PracticaFinalDistribuidos/ ficheros/pruebas.txt FILE TRANSFER OK </pre>	<pre> c> REGISTER miguel REGISTER OK c> CONNECT miguel CONNECT OK c> USERS CONNECTED USERS (2 users connected) OK miguel::172.28.94.212::35681 chen \::127.0.0.1::48499 c> DISCONNECT miguel DISCONNECT OK c> CONNECT miguel CONNECT OK c> MESSAGE 0 FROM chen hola que ase mondongo SENDATTACH chen Toma mi archivo /home/miguel_a/ Documents/UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt c> SENDATTACH chen Toma mi archivo /home/miguel_a/ Documents/UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt SENDATTACH OK - MESSAGE 1 c> FILE /home/miguel_a/ Documents/UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ </pre>

		<pre> c> DISCONNECT chen DISCONNECT OK c> UNREGISTER chen UNREGISTER OK </pre>	<pre> ficheros/test1.txt REQUESTED BY chen DISCONNECT miguel DISCONNECT OK c> UNREGISTER miguel UNREGISTER OK </pre>
2	Correc- ta ejecu- ción SEND y SEN- DATTACH cuando usuario destino no está conec- tado.	<pre> c> REGISTER chen REGISTER OK c> CONNECT chen CONNECT OK c> SEND miguel hola mensaje desconectado separado SEND OK - MESSAGE 0 c> DISCONNECT chen DISCONNECT OK c> CONNECT chen MESSAGE 1 FROM miguel Hola END FILE /home/miguel_a/ Documents/UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt CONNECT OK c> DISCONNECT chen DISCONNECT OK c> UNREGISTER chen UNREGISTER OK </pre>	<pre> c> REGISTER miguel REGISTER OK c> CONNECT miguel CONNECT OK MESSAGE 0 FROM chen hola mensaje desconectado separado c> SENDATTACH chen Hola / home/miguel_a/Documents/ UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ ficheros/test1.txt SENDATTACH OK - MESSAGE 1 c> DISCONNECT miguel DISCONNECT OK c> UNREGISTER miguel UNREGISTER OK </pre>
3	Probando error al rea- lizar GET- FILE debi- do a usua- rio desco- nectado	<pre> c> REGISTER adios REGISTER OK c> CONNECT adios CONNECT OK c> SENDATTACH hola buenas que tal /home/chenpower3/ Desktop/distribuidos/ final/ PracticaFinalDistribuidos/ ficheros/pruebas.txt SENDATTACH OK - MESSAGE 0 c> DISCONNECT adios DISCONNECT OK </pre>	<pre> c> REGISTER hola REGISTER OK c> CONNECT hola CONNECT OK c> MESSAGE 0 FROM adios buenas que tal END FILE /home/chenpower3/ Desktop/distribuidos/final/ PracticaFinalDistribuidos/ ficheros/pruebas.txt GETFILE adios /home/ chenpower3/Desktop/ distribuidos/final/ PracticaFinalDistribuidos/ ficheros/pruebas.txt /home/ miguel_a/Documents/ UC3M_Y3_C2/ SistemasDistribuidos/ PracticaFinalDistribuidos/ </pre>

			ficheros/test1.txt FILE TRANSFER FAIL, user not connected
--	--	--	--

3.1.2. Resultados en servidores

En este apartado presentamos los resultados obtenidos en las terminales de los distintos servidores que hay. La salida mostrada está asociada a los tests del apartado anterior.

ID	Terminal Servidor	Terminal Logger
1	<pre>s> init server 0.0.0.0:5000 s> s> REGISTER miguel OK s> CONNECT miguel OK s> REGISTER chen OK s> CONNECT chen OK s> CONNECTEDUSERS OK s> DISCONNECT miguel OK s> CONNECTEDUSERS OK s> CONNECT miguel OK s> SEND MESSAGE 000 FROM chen TO miguel s> SEND MESSAGE 001 FROM miguel TO chen s> CONNECTEDUSERS OK s> DISCONNECT miguel OK s> UNREGISTER miguel OK s> DISCONNECT chen OK s> UNREGISTER chen OK</pre>	<pre>miguel REGISTER miguel CONNECT chen REGISTER chen CONNECT miguel USERS miguel DISCONNECT chen USERS miguel CONNECT chen SEND miguel SENDATTACH chen USERS miguel DISCONNECT miguel UNREGISTER chen DISCONNECT chen UNREGISTER</pre>
2	<pre>s> init server 0.0.0.0:5000 s> s> REGISTER chen OK s> REGISTER miguel OK s> CONNECT chen OK s> MESSAGE 000 FROM chen TO miguel STORED s> CONNECT miguel OK s> DISCONNECT chen OK s> MESSAGE 001 FROM miguel TO chen STORED s> CONNECT chen OK s> DISCONNECT chen OK s> UNREGISTER chen OK s> DISCONNECT miguel OK s> UNREGISTER miguel OK</pre>	<pre>chen REGISTER miguel REGISTER chen CONNECT chen SEND miguel CONNECT chen DISCONNECT miguel SENDATTACH chen CONNECT chen DISCONNECT chen UNREGISTER miguel DISCONNECT miguel UNREGISTER</pre>
3	<pre>s> REGISTER hola OK s> CONNECT hola OK s> REGISTER adios OK s> CONNECT adios OK</pre>	<pre>hola REGISTER hola CONNECT adios REGISTER adios CONNECT</pre>

<pre>s> SEND MESSAGE 000 FROM adios TO hola s> DISCONNECT adios OK s> CONNECTEDUSERS OK</pre>	<pre>adios SENDATTACH adios DISCONNECT hola USERS</pre>
--	---

3.2. Pruebas de comprobación de errores

En este apartado, se realizan diversos tests para comprobar que se ejecutan los errores esperados al realizar una operación inválida.

ID	Explicación	Terminal cliente	Terminal Server	Terminal Logger
1	Register de usuario existente	<pre>c> REGISTER prueba REGISTER OK c> REGISTER prueba USERNAME IN USE</pre>	<pre>s> REGISTER prueba OK s> REGISTER prueba FAIL</pre>	<pre>prueba REGISTER prueba REGISTER</pre>
2	Unregister de usuario no existente	<pre>c> UNREGISTER hola USER DOES NOT EXIST</pre>	<pre>s> UNREGISTER hola FAIL</pre>	<pre>hola UNREGISTER</pre>
3	Connect cuando usuario no existe	<pre>c> CONNECT adios CONNECT FAIL, USER DOES NOT EXIST</pre>	<pre>s> CONNECT adios FAIL</pre>	<pre>adios CONNECT</pre>
4	Connect cuando usuario ya está conectado	<pre>c> REGISTER hola REGISTER OK c> CONNECT hola CONNECT OK c> CONNECT hola USER ALREADY CONNECTED</pre>	<pre>s> REGISTER hola OK s> CONNECT hola OK s> CONNECT hola FAIL</pre>	<pre>hola REGISTER hola CONNECT hola CONNECT</pre>
5	Disconnect cuando usuario no existe	<pre>c> REGISTER prueba REGISTER OK c> DISCONNECT NOT_EXIST DISCONNECT FAIL, USER DOES NOT EXIST</pre>	<pre>s> REGISTER prueba OK s> DISCONNECT NOT_EXIST FAIL</pre>	<pre>prueba REGISTER NOT_EXIST DISCONNECT</pre>
6	Disconnect cuando usuario no está conectado. PARTIENDO DEL TEST ANTERIOR	<pre>c> DISCONNECT hola DISCONNECT OK c> DISCONNECT hola DISCONNECT FAIL, USER NOT CONNECTED</pre>	<pre>s> DISCONNECT hola OK s> DISCONNECT hola FAIL</pre>	<pre>hola DISCONNECT hola DISCONNECT</pre>

7	Send cuando usuario no existe	c> SEND adios me recibes? SEND FAIL, USER DOES NOT EXIST	No aparece nada, en el protocolo de la guía no se pide que se imprima ningún error en el servidor en este caso	hola SEND
8	Users cuando usuario no está conectado	c> REGISTER test8 REGISTER OK c> USERS CONNECTED USERS FAIL	s> REGISTER test8 OK s> CONNECTEDUSERS FAIL	test8 REGISTER test8 USERS
9	SendAttach cuando usuario no existe	c> SENDATTACH noexisto Hola me recibes? /home/prueba.txt SENDATTACH FAIL, USER DOES NOT EXIST	No aparece nada, en el protocolo de la guía no se pide que se imprima ningún error en el servidor en este caso	test8 SENDATTACH

4. Conclusiones

Al comienzo de la práctica planteamos utilizar la misma estructura de servidor y cliente que usamos en el ejercicio 2. Sin embargo, nos cuestionamos si almacenar el estado del usuario sería más práctico con una lista de usuarios conectados o simplemente guardarlo en el struct.

Al final optamos por guardarlo en el struct debido a que en todas las funciones planteamos leer los archivos asociados a cada cliente. Nos pareció que mantener una lista de usuarios conectados podría ser redundante ya que de todos modos revisariamos cada struct guardado. Sin embargo, para almacenar la información de cada usuario conectado (actualizado al hacer USERS o GETFILE) acabamos utilizando un diccionario. De esta forma, podemos acceder rápidamente a los datos de cada individuo mediante su nombre de usuario. Concretamente, el diccionario que creamos posee la siguiente estructura:

[clave = nombre_usuario] => (ip, puerto)

Tuvimos problemas con la comunicación entre los clientes en Python. Esto es debido a que en un principio mandamos el mensaje de acción como `b"GET_FILE\0"`. Esto nos dio problemas debido a que el worker puede recibir hasta 24 bytes de información por las acciones de SEND y SENDATTACH. En Python si un socket escucha mediante `socket.listen()` espera recibir los bytes especificados. Por lo tanto, si el socket no recibía el número de bytes concretos, este se quedaba esperando. Tuvimos que asegurarnos que, independientemente de la cadena que se envíe al cliente Python, esta tenga un tamaño de 24 bytes para que pueda ser recibida correctamente. En C no tuvimos ese problema ya que al inicializar el buffer del mensaje con una cadena de caracteres concreta, la parte no ocupada del buffer se rellena con `\0`s. Por consiguiente, la función `sendMessage` mandaría la cantidad de bytes correcta. Seguimos la misma metodología al hacer `b'GET_FILE' + b'\0' * 16`, solucionando el problema.

En conclusión, esta práctica resultó muy útil para afianzar los contenidos dados a lo largo del curso. Pudimos entender mejor como se manejan los sockets para lograr una conexión entre cliente y servidor y cómo estos se intercambian mensajes entre sí. Además, la incorporación del servidor RPC nos permitió aprender como se puede conectar un servidor con otro, en este caso, importando el `proxy-rpc` y utilizándolo directamente en el servidor gestor de peticiones. Por último, la sección del servidor web, nos permitió visualizar brevemente cómo utilizar el protocolo HTTP para enviar y recibir peticiones.